

Kube-DC

A multi-tenant private cloud platform for infrastructure you own.

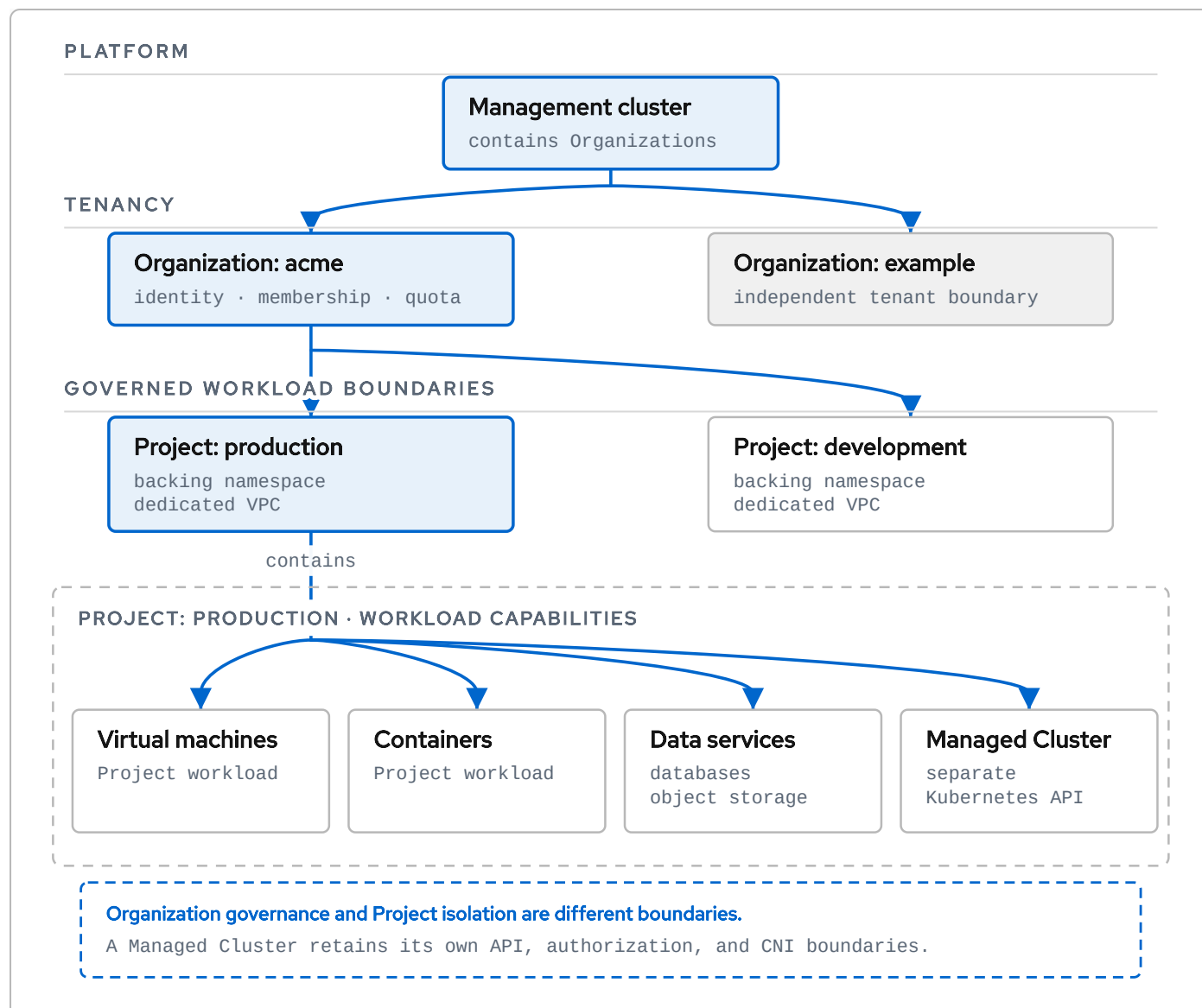
Kube-DC turns your servers into a private cloud your whole organization consumes as a service: departments and teams get self-service projects with their own networks, virtual machines, Kubernetes clusters, databases and storage – while your platform team keeps central control of capacity, security and cost. Your hardware, your datacenter, your data.

Who it's for

- **Public-sector organizations** that must keep workloads and data on infrastructure they control, with strict separation between departments.
- **Large enterprises and retailers** consolidating VMware-era virtual machines and modern containerized services onto one platform.
- **Universities and research institutions** giving faculties, labs and student groups isolated, quota-controlled environments on shared hardware.
- **Datacenter and hosting operators** selling cloud services to their own customers under their own brand.

At a glance

- **One platform for VMs and containers.** Virtual machines and Kubernetes workloads run side by side – same network fabric, same access model, same quota and billing.
- **Tenancy that reaches the network.** Every project gets its own VPC and subnet, not just a namespace label.
- **Self-service without losing control.** Teams provision VMs, clusters, databases, storage and public endpoints against quotas you set – through a web console, `kubect1`, a CLI, or an AI coding assistant.
- **Runs on standard x86-64 servers.** A platform starts at three nodes; capacity grows by adding nodes. Production sizing is validated against your workload in an architecture review.
- **Assembled from named open-source components.** Kubernetes, KubeVirt, Kube-OVN, Ceph, Keycloak, CloudNativePG and more – integrated and operated as one product.

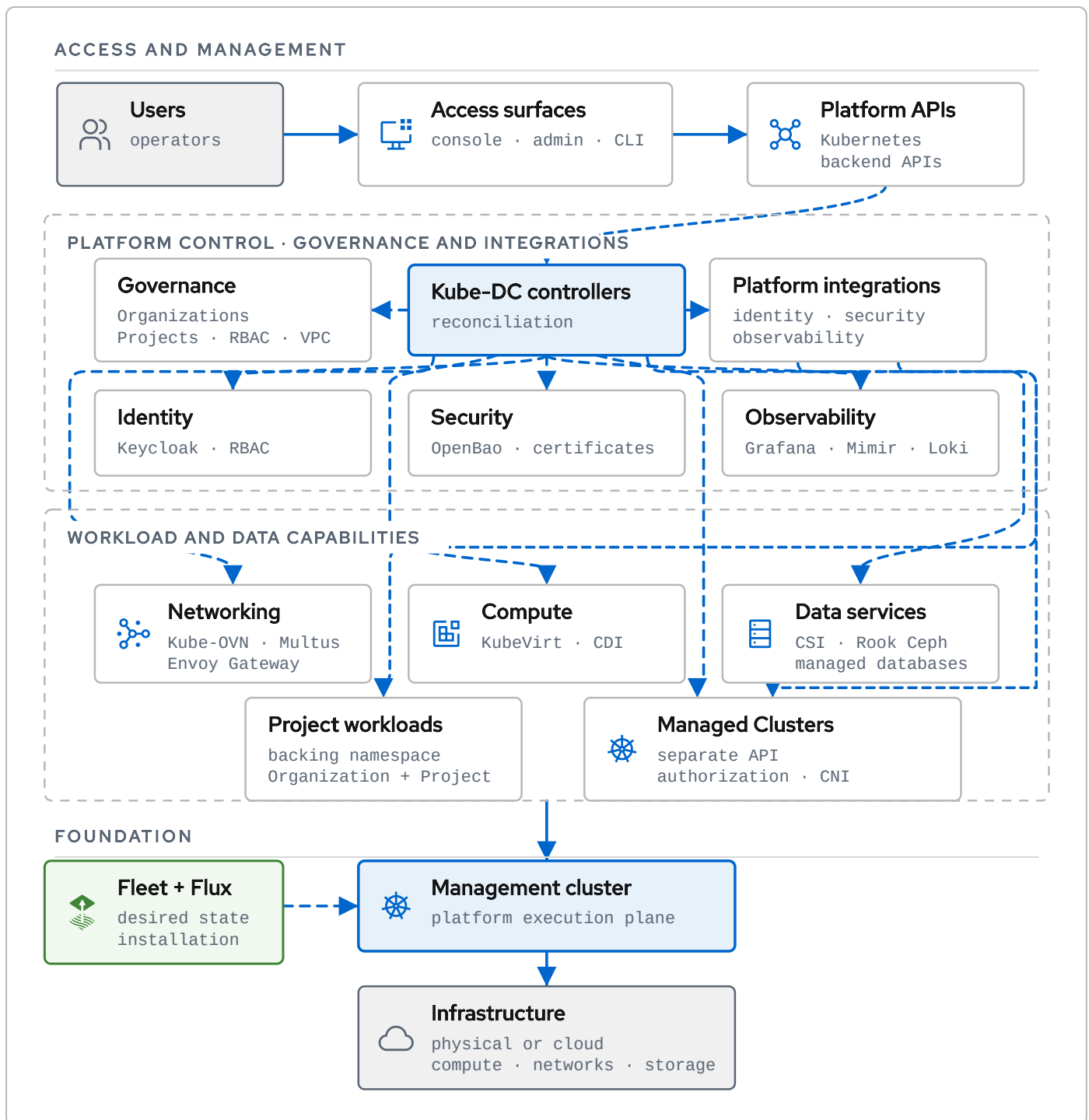


Kube-DC product hierarchy. Organizations are tenant boundaries; Projects are governed workload boundaries inside an Organization.

How it works

Kube-DC installs on a Kubernetes cluster running on your servers. Platform controllers reconcile a small set of resources – organizations, projects, machines, clusters, databases, keys – into the systems that do the work: software-defined networking, virtualization, identity, storage, ingress and observability. The result behaves like a public cloud, scoped to your hardware:

1. **Your platform team** installs Kube-DC with the `kube-dc` CLI, which bootstraps a GitOps repository – from then on, the platform's configuration is declarative and versioned.
2. **Organizations** are your tenants: a ministry's directorates, a university's faculties, a retailer's business units, or an operator's customers. Each organization has its own identity realm, members, plan and billing scope.
3. **Projects** are where teams deploy: each is an isolated environment with a namespace, RBAC, its own VPC and optional quotas – reachable through the console or a standard kubeconfig.



Kube-DC architectural layers. Organizations and Projects govern resources in the management cluster; Managed Clusters retain separate API, authorization, and CNI boundaries.

Multi-tenancy: organizations and projects

Layer	Scope	Mechanism
Identity	Organization	A dedicated identity realm per organization (Keycloak SSO – console, API, CLI)
Namespace & RBAC	Project	Hierarchical namespaces with project-scoped roles
Network	Project	A dedicated VPC and subnet per project
Quota	Project	Per-project quotas within the organization's plan
Billing & chargeback	Organization	Plans, usage and subscription billing per organization

Organization quotas constrain the configured Kubernetes resource dimensions – CPU, memory, storage, pods, load balancers – and optional project quotas subdivide them; object-storage and physical capacity have their own controls and capacity planning. Project access is a curated role set for deploying and operating workloads; cluster-scoped administration stays with the platform team, and the supported project roles omit interactive pod `exec/attach`, enforced by admission policy (one-off tasks run as auditable Jobs instead).

KUBE-DC

1

Volodymyr Tsap

shalb

Projects

Users

Organization Groups

Project Roles

Billing

Audit Logs

Settings

Observability

Projects

Manage organization projects and their configurations. View project details, resource usage, and settings.

Create New Project

Name	Display ...	CIDR Block	Status	VLANs	Pods	Resource Quotas	Created	Actions
docs	docs	10.0.0.0/16	Ready	–	3 running 4 total	CPU 5.0/42.0 Mem 11.1Gi/108.0Gi Stor 75.3Gi/580.0Gi Pods 3/500	22/02/2026 15:44:55	Go to Project Details Delete
e2e	e2e	10.77.0.0/16	Ready	–	1 running 1 total	CPU 12.0/42.0 Mem 51.0Gi/108.0Gi Stor 93.1Gi/580.0Gi Pods 1/500	04/07/2026 19:20:00	Go to Project Details Delete
jumbolot	jumbolot	10.0.0.0/16	Ready	–	17 running 21 total	CPU 13.9/42.0 Mem 24.1Gi/108.0Gi Stor 292.4Gi/580.0Gi Pods 17/500	26/01/2026 00:00:12	Go to Project Details Delete

Organization administrators see Project readiness, network allocation, running workloads and quota consumption in one view.

Self-service projects: deploy applications directly

A project comes with a kubeconfig. Teams deploy applications, Helm charts, Jobs and services straight into it – there is no cluster to request first and no ticket queue. A complete stateful stack – application pods, a managed database, S3 bucket, shared volumes, HTTPS endpoint, autoscaling – deploys from plain Kubernetes manifests, with the

platform providing the ingress, certificates, database engine and storage underneath.

When a team needs cluster-scoped control – operators, CRDs, custom controllers – it provisions a managed Kubernetes cluster instead (below).

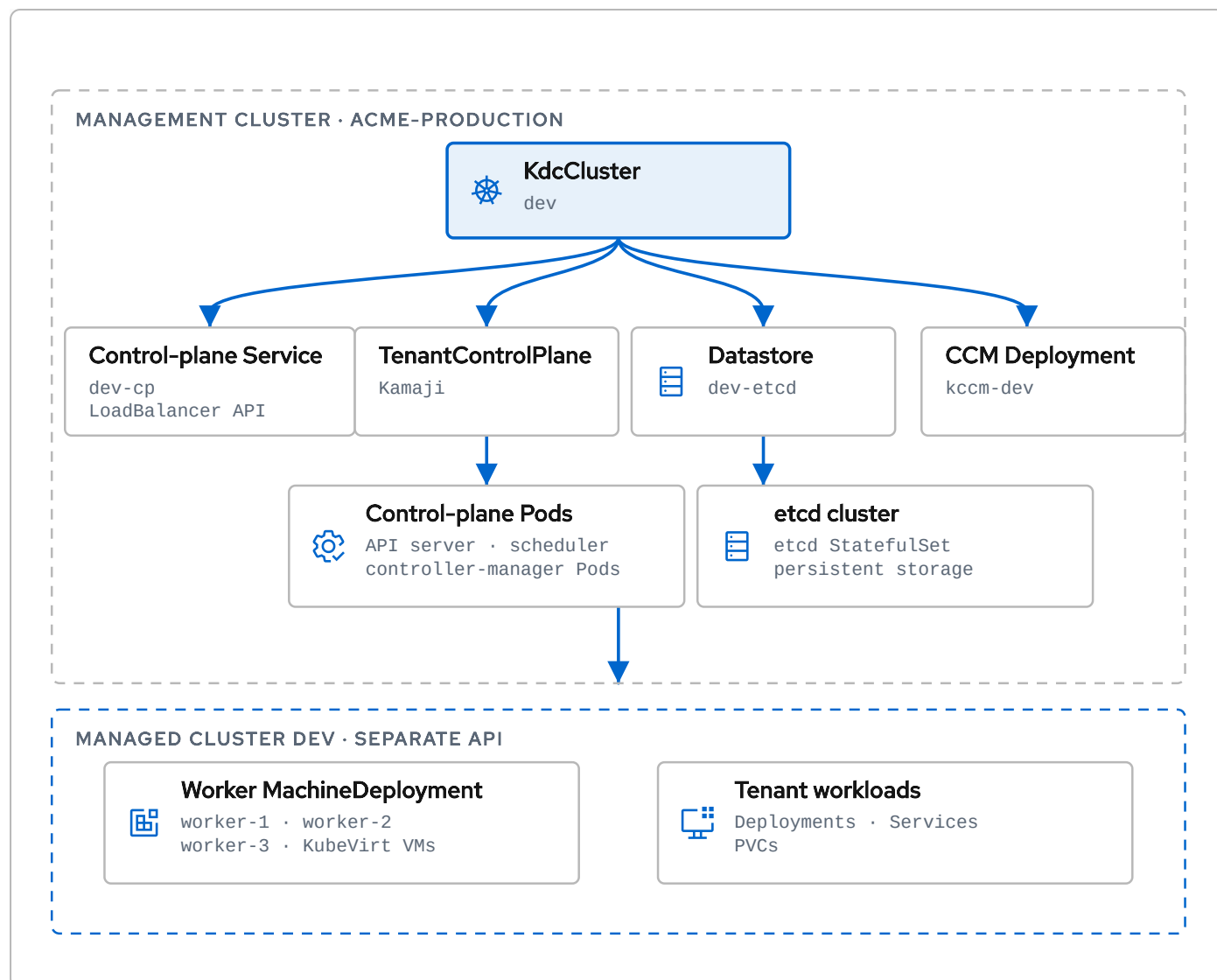
Virtual machines

- Linux and Windows guests from a prepared-image catalog – new VMs clone a prepared image snapshot, with no per-VM image download; bring your own images, with guest compatibility validated on import.
- Cloud-init configuration, SSH-key injection, interactive console in the web UI.
- Snapshots of running VMs; disk, CPU and memory sizing per VM.
- **Live migration:** VMs on the Migratable profile (shared block storage) are eligible to move between CPU-compatible hosts with sufficient capacity – the mechanism behind draining a host for maintenance.
- VMs attach to the project's VPC like any other workload: same networks, same floating IPs, same load balancers as containers.

Managed Kubernetes clusters

Full tenant-administered Kubernetes clusters, provisioned from a project with one manifest:

- **Hosted control planes** run as managed pods on the platform – tenants get a tenant-administered Kubernetes API with per-cluster PKI, without operating masters.
- **Worker pools** sized per pool (CPU, memory, disk, image), with autoscaling bounds per pool.
- **Control-plane and etcd vertical autoscaling** on by default – API server and etcd resources are right-sized within configured bounds as cluster load grows.
- **Staged, tenant-controlled upgrades** – control plane and worker pools move separately, in steps.
- **Scheduled etcd snapshots** (with optional envelope encryption) and restore – self-service for single-replica etcd datastores, assisted for multi-replica; private in-VPC endpoints by default, public HTTPS endpoints via the platform's gateway and certificate issuer on request.
- Admin and break-glass kubeconfigs retrievable by the tenant.



KdcCluster owns a Kamaji control plane, datastore, worker MachineDeployment, and CCM inside the Project infrastructure namespace; users reach a separate tenant Kubernetes API and run workloads on its workers.

Managed databases

- PostgreSQL and MariaDB, provisioned by manifest or console.
- Multi-replica engine replication; after an eligible primary failure an available replica is promoted automatically – behavior depends on your deployment's topology and capacity.
- Scheduled and on-demand backups to the project's S3 bucket, envelope-encrypted when a KMS key is configured; restore into a new database or in place.
- Auto-generated credentials delivered as Kubernetes Secrets; optional credential-rotation policies.

GPU services

- **Shared GPU for containers:** several workloads on one physical card, each holding a catalogued fixed slice of a specific GPU model – inference, notebooks, small training runs. The memory slice is enforced; the compute share is cooperative, so it is a density mechanism rather than a performance or security guarantee.
- **Dedicated GPU VMs:** one whole device attached to one guest, for workloads needing a stronger boundary (not live-migratable).

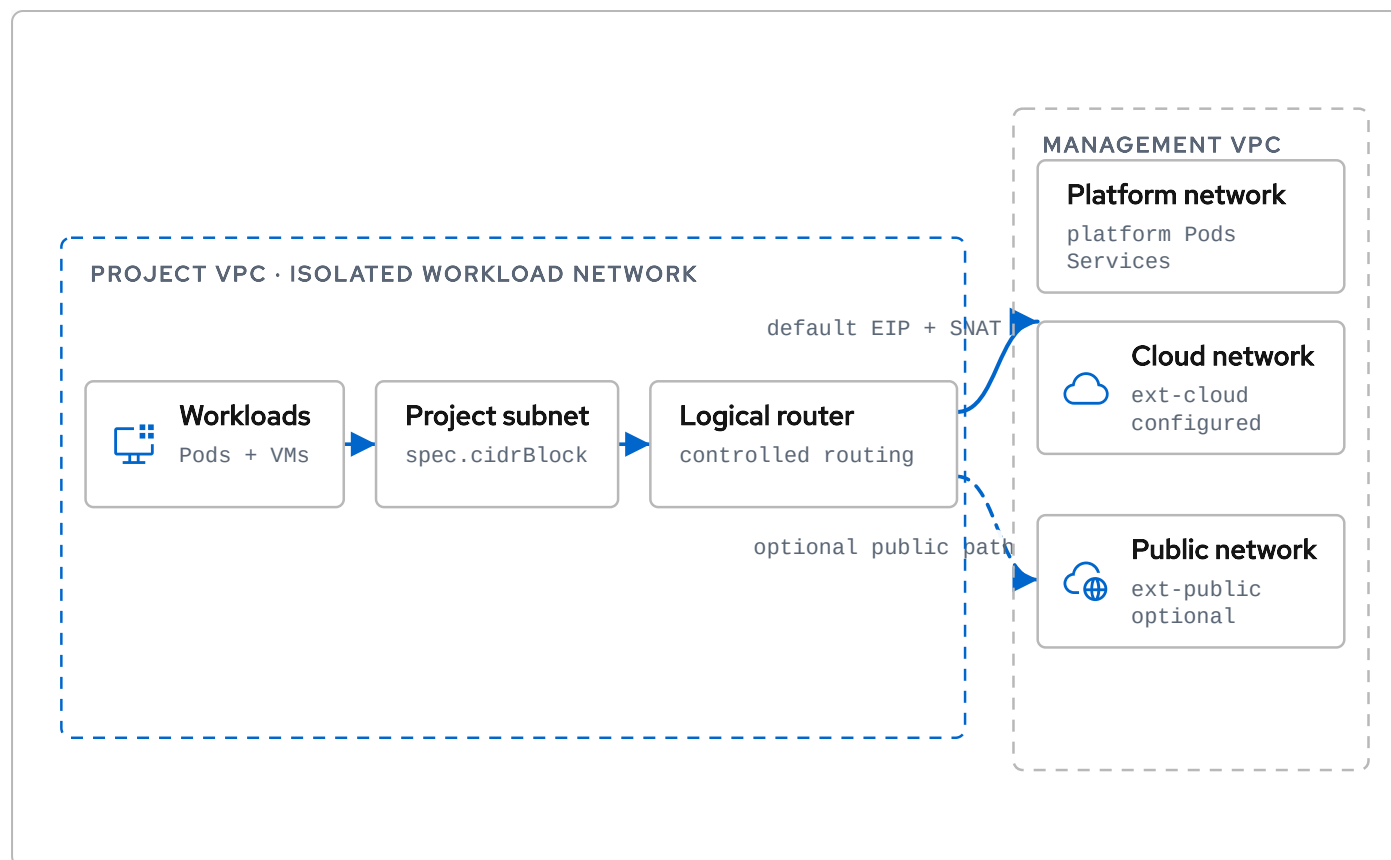
- **GPU as a governed product:** per-model entitlements enforced as quota, operator-held capacity reservations for organizations that can't queue, holder-safe node-mode transitions, an upgrade gate that qualifies device/kernel/driver/operator as one tuple, and a published threat and supply-chain model.
- GPU capabilities are enabled per cluster once your hardware, drivers and component set are qualified – GPU nodes couple those versions together, so enablement follows qualification.

Storage

- **Block storage** for VMs and workloads (Ceph RBD), including a shared read-write-many tier for multi-pod volumes and live-migratable VMs.
- **S3-compatible object storage** with per-project buckets, provisioned through a standard `ObjectBucketClaim`.
- **Volume snapshots** and instant clones from golden images.
- Storage classes, replication factors and capacity are yours to define – the platform runs on the disks and topology you give it.

Networking

- **A VPC per project** with private subnets; no private cross-project route exists by default – reachability exists only where a service is deliberately exposed.
- **External and floating IPs:** shared egress per project, dedicated addresses per load balancer, floating IPs that map to individual VMs – allocated from address pools your platform team defines.
- **LoadBalancer services and HTTPS ingress:** one annotation on a Service publishes it at a hostname with TLS, using the platform's gateway and certificate issuer on your address pools and DNS.
- **Physical VLAN attachment:** where the datacenter fabric trunks delegated VLANs to the platform nodes, a project's network bridges onto an existing VLAN at layer 2 – lab equipment, legacy systems, dedicated links – allocated from per-organization VLAN pools.
- Egress control and allowlists, operated by the platform team.



Each Project has its own VPC and creator-supplied workload subnet. Its default EIP and SNAT use the selected external network; public EIPs, FIPs, and Services are optional explicit paths.

Security and identity

- **Single sign-on** across console, API and CLI, with an identity realm per organization (Keycloak; OIDC).
- **Role-based access** per project from a curated, supported role set; custom roles under platform-team control.
- **Key management service:** per-project, purpose-scoped encryption keys backed by non-exportable keys in the platform's secrets backend (OpenBao).
- **Secrets manager** for application credentials, with sync into Kubernetes Secrets – keep secrets out of Git.
- **Managed certificates:** public trust via ACME or your organization's private CA.
- Admission policies enforce tenant boundaries at the API – including the platform-wide block on interactive pod access.

Observability

- **Configured automatically per organization.** When an organization is created, the platform's controllers provision its observability – a Grafana organization of its own, pre-built dashboards, a metrics tenant and log routing on the bundled multi-tenant Grafana/Mimir/Loki stack. Teams open Grafana and see their workloads; nothing to install.
- Metrics and logs are separated per tenant at the data layer – each organization sees only its own, including the control-plane telemetry of its managed Kubernetes clusters. Coverage and retention are operator-defined.
- Platform-level monitoring for the operator: cluster health, storage, networking and capacity, with alerting.

Billing and chargeback

Organizations subscribe to plans that define their capacity; usage is enforced as quota and visible per organization – the basis for internal chargeback between departments or for invoicing external customers. Subscription billing integrates with payment providers for operators selling the platform as a service.

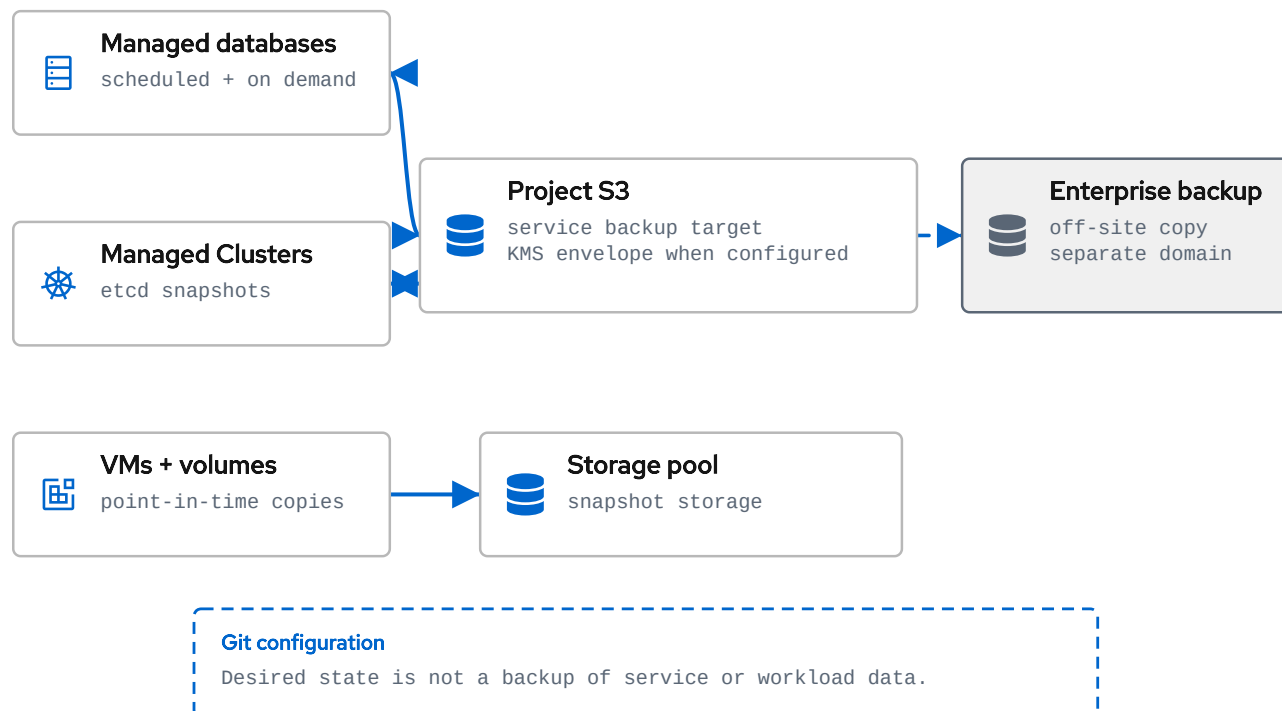
Automation and integration

- Tenant-facing services are Kubernetes-native or custom resources – the documented console and CLI workflows drive the same APIs, so `kubect1` and GitOps pipelines can too.
- The `kube-dc` CLI handles login, context switching and platform bootstrap.
- **Published agent skills** let AI coding assistants (Claude Code, Cursor and others) create projects, deploy applications, provision VMs and databases through guarded, documented workflows.

Data protection

Each managed service carries its own protection: database backups to project S3 (envelope-encrypted when a KMS key is configured); managed-cluster etcd snapshots and restore; VM and volume snapshots on demand. Platform configuration lives in Git as desired state – it is not a backup of service or workload data. For copies that must survive site or storage loss, integrate the platform's S3 endpoints and snapshot APIs with your enterprise backup system – Kube-DC does not replace it.

BACKUP SCOPE FOLLOWS THE DATA OWNER



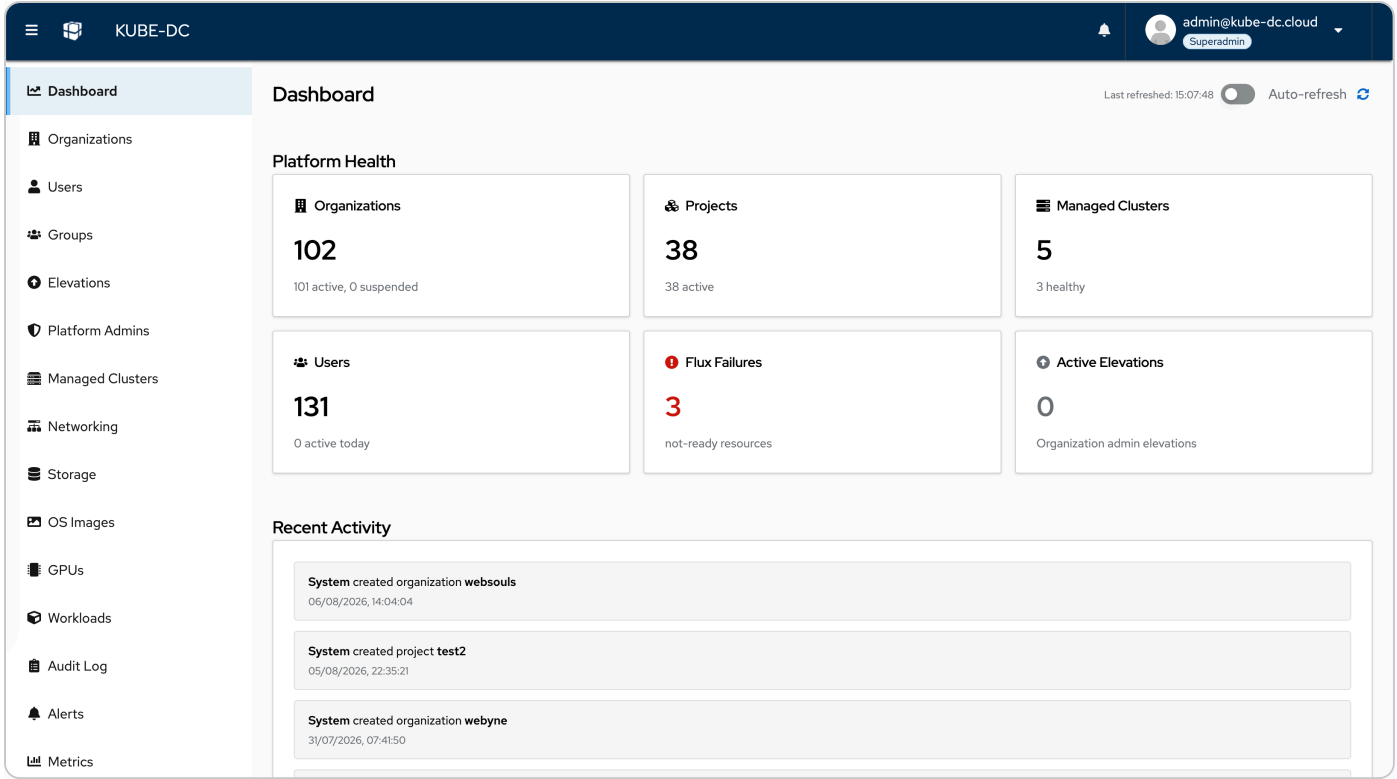
Each service protects its own data class: databases and Managed Cluster etcd use Project S3, VM and volume snapshots stay in the storage pool, and an enterprise backup integration is required for copies outside the platform's storage failure domain.

Operating the platform

Your platform team runs Kube-DC through three surfaces that cover the whole lifecycle:

- **Administration console** – a dedicated web panel for platform administrators: organizations and their plans, billing and subscriptions, capacity and storage health.
- **GitOps** – platform configuration is declaratively reconciled from a Git repository (Flux): component versions, platform settings and day-2 changes land as version-controlled commits.
- **kube-dc CLI** – carries the platform from installation through day-2: bootstrap of the GitOps repository, status and configuration commands, adoption of existing clusters, and login/context management for daily `kubectl` work.

Operator-side monitoring and alerting come from the same bundled observability stack the tenants use (see Observability).



The administration dashboard gives platform operators a consolidated health and activity view across the tenant estate.

Deployment and requirements

Reference baseline – subject to architecture validation. Production sizing and supported hardware depend on workload, storage topology, failure domains and validated NIC/firmware compatibility.

	Evaluation	Production
Server nodes	3	3+ (scale by adding nodes)
CPU per node	8 cores	16+ cores
RAM per node	32 GB	64+ GB
Storage per node	500 GB SSD	Per your capacity plan; dedicated disks for Ceph
Operating system	Ubuntu 24.04 LTS	Ubuntu 24.04 LTS
Network	VLAN-capable NIC; management, cloud and provider networks	Redundant NICs

Installation is CLI-driven: `kube-dc bootstrap init` scaffolds the GitOps repository and hands the platform to Flux for reconciliation. Day-2 operations – upgrades, configuration changes, component versions – are Git commits.

Platform components

Kube-DC integrates named upstream open-source components: Kubernetes, KubeVirt (virtualization), Kube-OVN (SDN), Rook-Ceph (storage), Keycloak (identity), Kamaji + Cluster API (hosted control planes), CloudNativePG and the MariaDB operator (databases), OpenBao (keys and secrets), Envoy Gateway and cert-manager (ingress and TLS), Prometheus/Mimir/Loki/Grafana (observability), Flux (GitOps). Component versions are pinned per Kube-DC release. Workloads built on standard Kubernetes objects and standard VM guest formats carry no platform-specific dependencies unless they use the platform's own resources – which are enumerated, not hidden.

Next steps

- **Evaluate:** install on three servers, or start in a hosted evaluation environment.
- **Talk to us:** architecture review against your hardware, network and tenancy requirements.

Function datasheets in this guide

This document is the overview; each major function has its own datasheet with full technical depth:

Function	Datasheet
Managed Kubernetes clusters	function-managed-kubernetes.md
Virtual machines	function-virtual-machines.md
Managed databases	function-managed-databases.md
Networking & VLAN attachment	function-networking.md
Storage & object storage	function-storage.md
Security, identity & keys	function-security.md
Observability	function-observability.md
GPU services	function-gpu.md

Managed Kubernetes Clusters

Give every team a real Kubernetes cluster – without giving every team a control plane to operate.

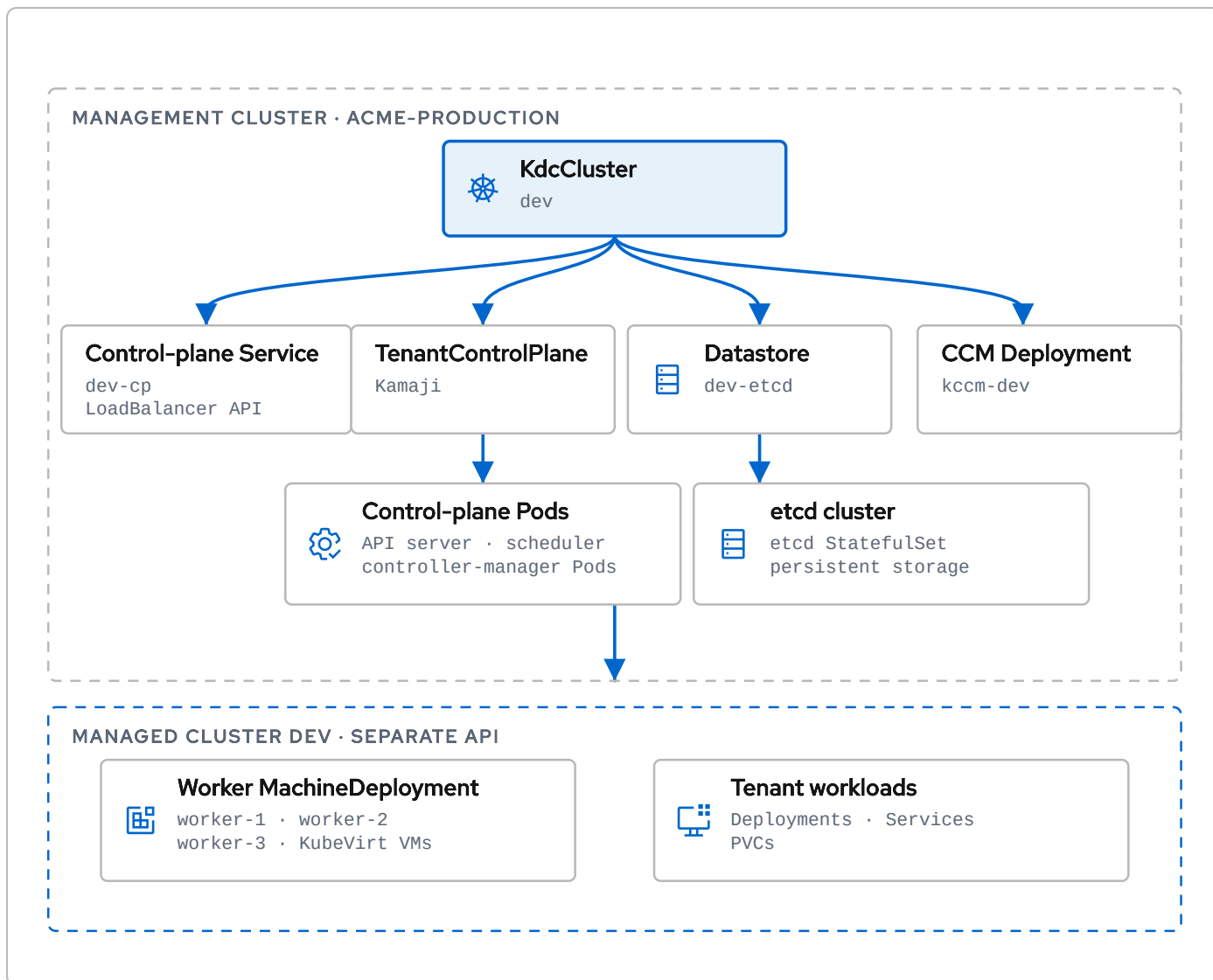
Kube-DC's managed clusters let your tenants – departments, faculties, customers – provision full, tenant-administered Kubernetes clusters from their projects. The platform runs the control planes; tenants own their clusters' workloads, upgrades and access. Your platform team operates one system, not one cluster per team.

1. Architecture

A managed cluster has two halves:

- **The hosted control plane** runs as managed pods on the platform cluster – API server, controller manager, scheduler – with a dedicated PKI per cluster. Tenants administer a full Kubernetes API without touching a master node. Control planes live on a platform-internal network that has no route from tenant networks; tenants reach only the API endpoints published to them.
- **Worker pools** run as virtual machines inside the tenant project's own VPC – the same isolated network that holds the project's other workloads. Workers join the control plane automatically at creation and on scale-up.

An optional **dedicated etcd datastore** per cluster (three replicas by default) keeps a tenant's cluster state on its own instance, with platform-managed certificate rotation. Clusters can alternatively share a platform datastore.



KdcCluster owns a Kamaji control plane, datastore, worker MachineDeployment, and CCM inside the Project infrastructure namespace; users reach a separate tenant Kubernetes API and run workloads on its workers.

2. Provisioning

A cluster is one manifest (or a console form):

```
apiVersion: k8s.kube-dc.com/v1alpha1
kind: KdcCluster
metadata:
  name: analytics
  namespace: acme-data-platform
spec:
  version: v1.36.1
  controlPlane:
    replicas: 1 # 2+ = multi-replica topology; availability depends on deployment
  datastore:
    dedicated: true # own etcd instance for this cluster
  network:
    serviceCIDR: 10.96.0.0/16
    podCIDR: 10.244.0.0/16
  workers:
```

```
- name: general
  replicas: 3
  cpuCores: 4
  memory: 8Gi
  diskSize: 30Gi
```

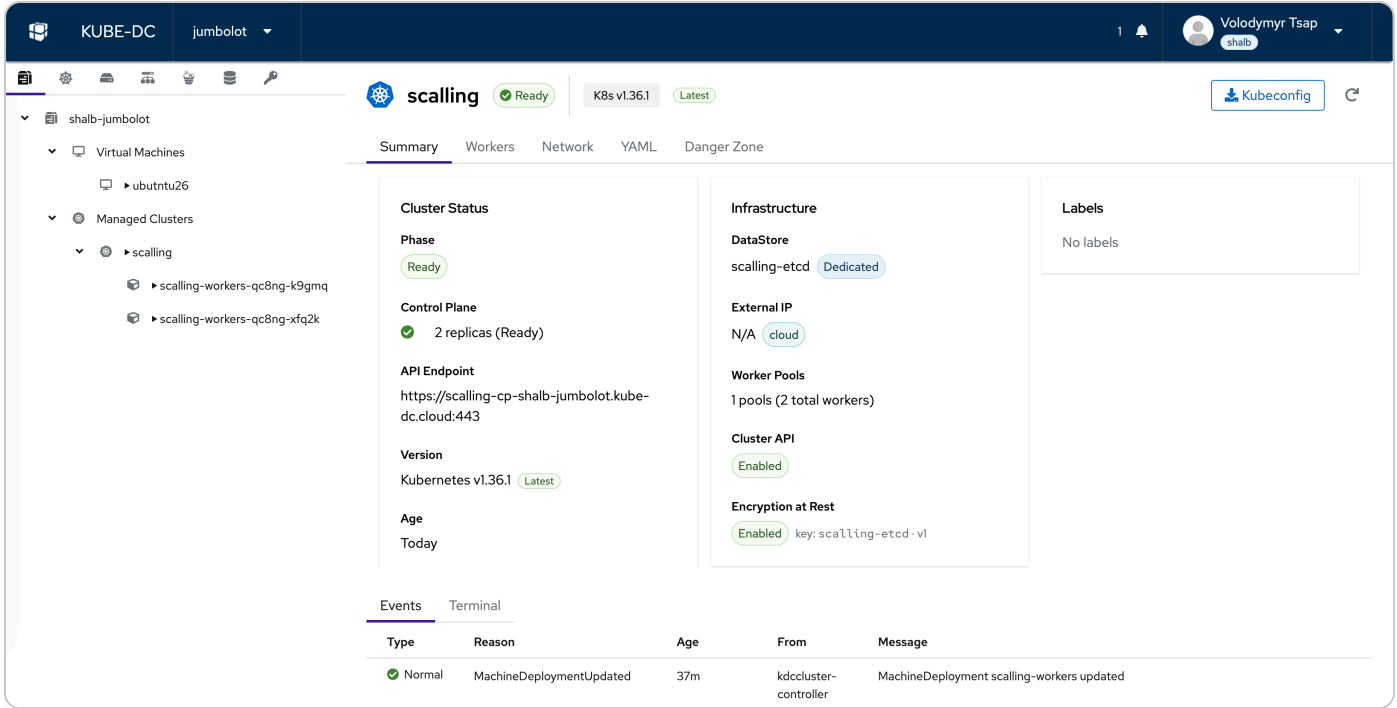
The platform provisions the control plane, datastore, worker VMs, cluster PKI and kubeconfigs, and joins the workers – no further steps. Deleting the `KdcCluster` tears the cluster down; backup-object lifecycle follows the configured bucket and retention policy.

3. Endpoints and access

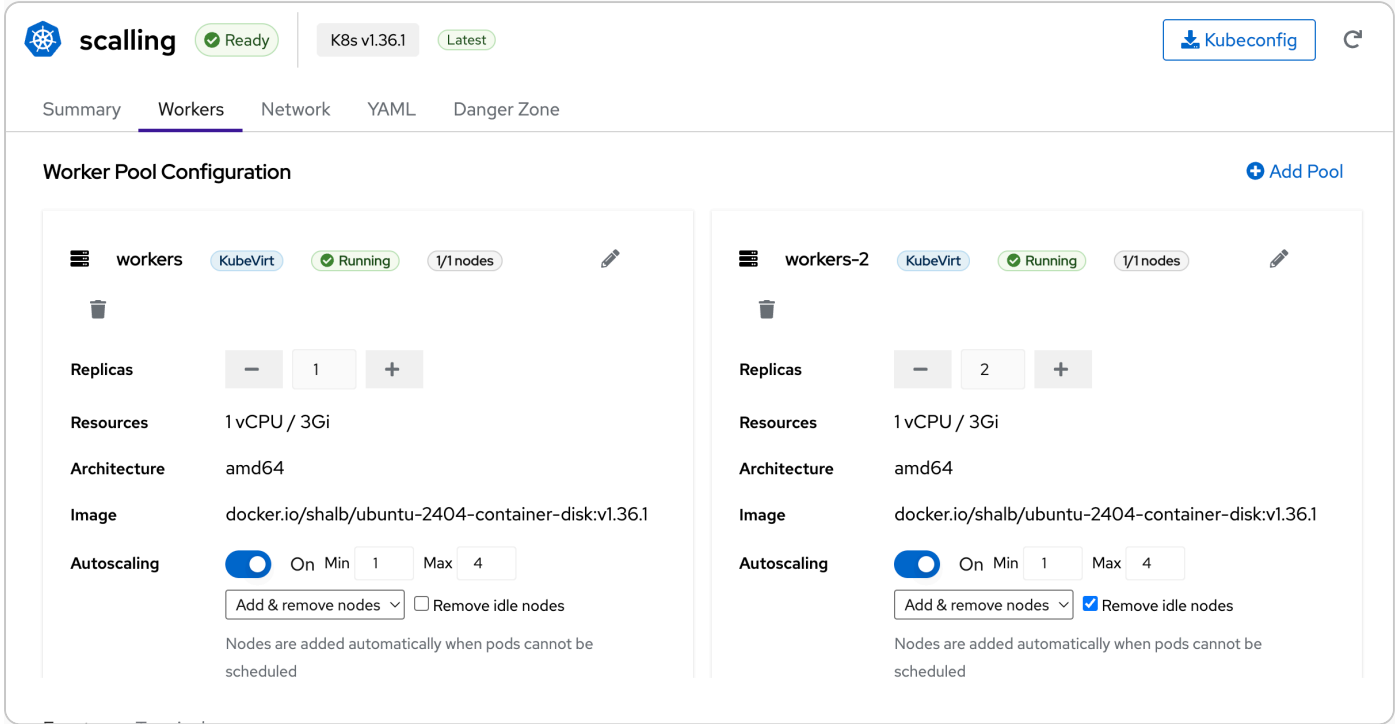
- **In-VPC endpoint by default** – the cluster API is reachable from the owning project's network.
- **Public HTTPS endpoint on request** – one annotation publishes the API at a stable hostname, using the platform's gateway, DNS and certificate issuer; the API server certificate is re-issued to include the public name without manual PKI work.
- **Kubeconfigs as Kubernetes Secrets** in the owning project: an admin kubeconfig for the in-VPC endpoint and an external kubeconfig for the public one. Retrieval works from the console, the CLI, or `kubect l` – so CI systems can fetch cluster credentials the same way tenants do.
- **Break-glass access** – the admin kubeconfig authenticates independently of the platform's SSO; access still depends on endpoint, network and control-plane availability.

4. Worker pools

- Sized per pool: CPU, memory, disk, OS image, replica count; multiple pools per cluster for heterogeneous workloads.
- Worker OS images ship as prepared, versioned images matched to the Kubernetes version.
- **Per-pool autoscaling** with `minReplicas`/`maxReplicas` bounds and a maximum step per scaling event.
- Rolling operations respect `PodDisruptionBudgets` during drains, within a configurable drain timeout.



The cluster summary brings endpoint access, control-plane readiness, worker capacity and security status into one tenant-facing view.



Worker-pool autoscaling is configured independently per pool, including minimum and maximum replicas and whether idle nodes may be removed.

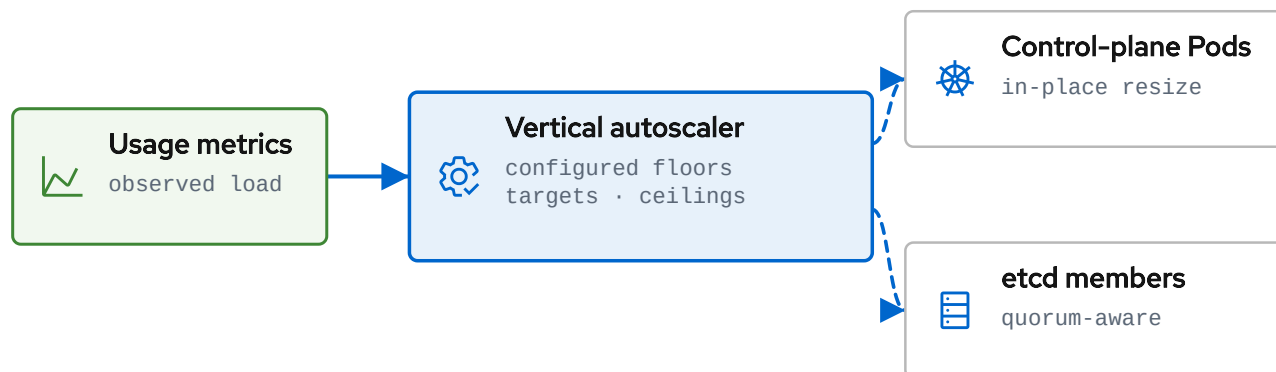
5. Control-plane and etcd right-sizing

Tenant control planes are right-sized automatically, within configured bounds and available platform capacity. Both the control-plane pods and the etcd members carry VerticalPodAutoscalers, enabled by default:

- Resource targets follow actual load between configured floors and ceilings – small clusters stay small; growing clusters get API-server and etcd headroom automatically.

- Scaling is quorum-aware by design: recommendations are applied in-place where possible, and etcd resizing respects member quorum. Single-replica datastores use a conservative initial-assignment mode.

BOUNDED AUTOMATIC RIGHT-SIZING



Usage metrics feed bounded recommendations to the control plane and datastore; the platform applies them in place where possible and preserves etcd quorum.

6. Upgrades

Upgrades are staged and tenant-controlled:

1. The tenant (or platform team, by arrangement) sets the target version.
2. The **control plane upgrades first**, without replacing worker nodes.
3. **Worker pools roll in steps** – new nodes join, old nodes drain and leave, pool by pool, with PodDisruptionBudgets honored during drains.

Control plane and workers move separately, so tenants can pause between stages, validate, and continue. Supported version skew follows upstream Kubernetes policy.

STAGED · TENANT-CONTROLLED UPGRADE



The tenant selects a supported target; the hosted control plane moves first, worker pools roll in bounded steps, and the tenant can validate between stages.

7. Snapshots and restore

- **Scheduled etcd snapshots** are enabled automatically when the project's backup bucket exists (daily by default; schedule and retention configurable per cluster), written to the project's S3-compatible storage.
- **On-demand snapshots** at any time – from the console or by triggering the snapshot job.
- **Optional envelope encryption** of snapshots with a tenant-scoped key reference; without it, snapshots are stored unencrypted.
- **Self-service restore**: pick a snapshot, trigger the restore, and the control plane returns with the cluster's API state rolled back to the snapshot. The tenant API is unavailable during restore.

Scope and current limits, stated plainly:

- Etcd snapshots protect **Kubernetes API state**; they do not protect PersistentVolume contents – protect application data through its owning service (database backups, volume snapshots, object storage).
- Restore currently supports **single-replica etcd datastores**; multi-replica restore is on the roadmap. For clusters using the default three-replica dedicated datastore, restore is an assisted operation until then.

8. Isolation and security

- Control planes run on a platform-internal network with no route from tenant networks; workers and workloads live in the tenant project's VPC.
- Each cluster has its own PKI – certificates are per-cluster, rotated by the platform.
- Inside their cluster, tenants hold full cluster-admin: their own RBAC, CRDs, operators and admission configuration, independent of the platform's project roles.
- The cluster inherits the project's network boundary: reaching a cluster's workloads from outside happens only through the exposure the tenant configures (LoadBalancer services, HTTPS routes).

9. Observability

With the platform's observability stack deployed, control-plane logs and events of managed clusters surface in the owning organization's Grafana – tenants watch their API servers and controllers without filing a ticket, and the platform team monitors all control planes centrally. Coverage and retention are operator-defined.

10. Responsibilities – managed Kubernetes clusters

Concern	Your platform team (via Kube-DC)	Tenant
Control-plane operation, PKI, datastore	☐	–
Control-plane/etcd sizing	☐ automatic, within configured bounds	–
Worker pool sizing and autoscaling bounds	–	☐
Kubernetes version and upgrade timing	Controllers execute the staged rollout	☐ selects target and timing from supported versions
Etcd snapshots	Controllers execute scheduled snapshots	☐ policy, on-demand runs, restore initiation
Workloads, in-cluster RBAC, CRDs, operators	–	☐
Application data protection	–	☐ via managed services
Network exposure of cluster workloads	Platform provides EIPs/routes	☐ configures

Virtual Machines

Run the workloads you already have – beside the ones you're building next.

Kube-DC treats virtual machines as first-class workloads: they live in the same projects, attach to the same networks, obey the same quotas and appear on the same dashboards as containers. Legacy line-of-business systems, databases that ship as appliances, Windows servers – compatible workloads consolidate onto the platform without a separate virtualization stack to operate.

1. Guest support

- **Linux:** the prepared-image catalog identifies the validated distributions and versions (Ubuntu, Debian and other cloud-image distributions).
- **Windows:** provisioned from prepared images containing the required VirtIO drivers and QEMU guest agent, via the platform's Windows image pipeline – see the published Windows documentation.
- **Bring your own image:** import qcow2/raw disk images from HTTP or S3 sources into a project; guest and driver compatibility validation is yours.

2. Provisioning: prepared images and instant clones

Each project carries a catalog of prepared OS images published as storage snapshots. Creating a VM clones the snapshot – the root disk is created without a per-VM image download or data copy; completion depends on your hardware, storage and current load. VMs are declared as manifests (or created in the console):

- CPU cores, memory and disk size per VM; resizable as the workload grows (disk grows only).
- **Cloud-init** for first-boot configuration: packages, users, scripts.
- **SSH-key injection** from the project's managed keypair – no password distribution.
- **Interactive console** in the web UI for installation, rescue and break-glass access.

Create VM

Specify params

1

Step 1

2

Review

Subnet *

default

Root Storage size (GB) *

Max: 320 GB

20

Root disk storage *

Local disk (default)

Fast durable writes, node-local. No snapshots, no live migration.

Shared RBD

Shared Ceph-backed storage – supports snapshots and live migration. Slower durable writes.

Provisioning: Prepared image (Ubuntu 24.04 LTS) – fast clone from a golden snapshot

Availability

Enable live migration

Allows this VM to move during node maintenance. Requires shared block storage and a compatible CPU pool. Uses RWX Block mode. Migration needs temporary extra CPU quota.

Root disk

Shared RBD

Provisioning

Prepared image (min 27 GB)

Snapshots

Supported

Live migration

Not available

Tradeoff

Slower durable writes

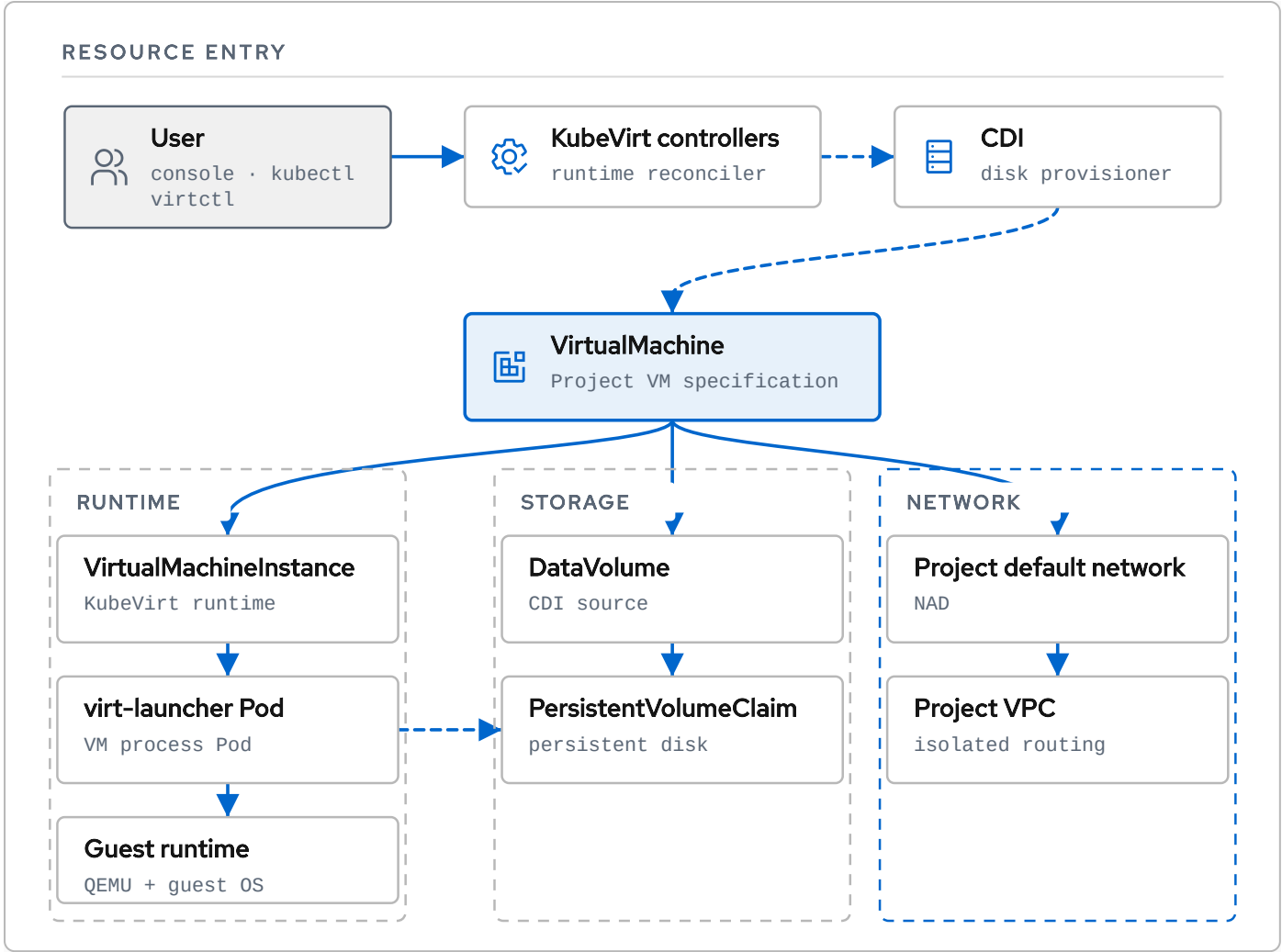
Back

Next

Cancel

VM creation makes storage durability and mobility trade-offs explicit before provisioning, including prepared-image cloning and the shared-storage requirement for live migration.

21 / 44



KubeVirt reconciles the VM runtime, CDI provisions the persistent disk, and Kube-OVN attaches the VM to its Project VPC; all branches originate from the Project VirtualMachine resource.

3. VM profiles: standard and migratable

Two documented profiles, selected at creation:

	Standard profile	Migratable profile
Root disk	Node-local or replicated block	Shared block storage (read-write-many)
Host maintenance	VM restarts on another host	Eligible for live migration during host drains
Typical use	Stateless or restart-tolerant workloads	Long-lived servers, stateful systems in VMs

The Migratable profile pins a common CPU model across the migration pool. A VM is eligible for live migration when its disks use the configured shared block tier and a CPU-compatible destination host has sufficient capacity; failed or ineligible migrations may require a restart.

4. Lifecycle and operations

- Start, stop, restart and delete from the console, `kubect l` or the CLI.
- **Snapshots of running VMs** – online snapshots via the standard Kubernetes snapshot API; install the QEMU guest agent for filesystem-consistent captures.
- Eviction behavior per VM: live-migrate or restart on drain.
- VM manifests are declarative Kubernetes resources – keep them in Git like the rest of your infrastructure.

Guest OS



Launch Remote Console

Launch SSH Terminal

The VM detail view provides browser-based console and SSH entry points for installation, rescue and break-glass access.

5. Networking

VMs attach to the project's VPC like any pod:

- Private addresses in the project subnet; reachable from the project's other workloads.
- **Floating IPs** give a VM its own external address – inbound and outbound – without a load balancer.
- LoadBalancer services and HTTPS routes can front VM-hosted services exactly as they front pods.
- With **VLAN attachment**, VMs talk layer-2 to existing datacenter equipment – the consolidation path for systems that expect to sit on a specific network segment.

6. Storage

- Root and data disks as block volumes; disk expansion online.
- Additional volumes attach and detach as Kubernetes resources.
- Volume snapshots and clones for copies, templates and test environments.

7. GPU guests

A VM can be given a whole physical GPU (dedicated passthrough) where the platform team has enabled and qualified GPU support – a stronger boundary than software GPU sharing, at the cost of live migration. See the [GPU services datasheet](#) for products, entitlements and the qualification model.

8. Responsibilities – virtual machines

Concern	Your platform team	Tenant
Virtualization layer, hosts, live-migration machinery	☐	–
Prepared image catalog	☐ publishes	☐ may bring own images
VM sizing, lifecycle, in-guest configuration	–	☐
Guest OS patching and hardening	–	☐
VM data protection	Snapshot APIs provided	☐ snapshots + in-guest/enterprise backup
Network exposure	IPs and routes provided	☐ configures

8. What to know before migrating

Disk import from other virtualization platforms is supported (qcow2/raw); migration itself is a planned operation – inventory, guest preparation (VirtIO drivers for Windows), disk conversion, cutover – not an automatic "lift". VMs do **not** need to be refactored into containers: the point of this function is that they run as VMs, on the same platform your containerized services use.

Managed Databases

Managed PostgreSQL and MariaDB for every team – without each tenant operating a database engine.

Tenants provision databases the way they provision everything else on the platform: a short manifest or a console form. The platform's controllers run the engines, replication and backups; teams get an endpoint and a credentials Secret.

1. Engines and provisioning

PostgreSQL and MariaDB, in the versions published by the platform's catalog. A database is one resource:

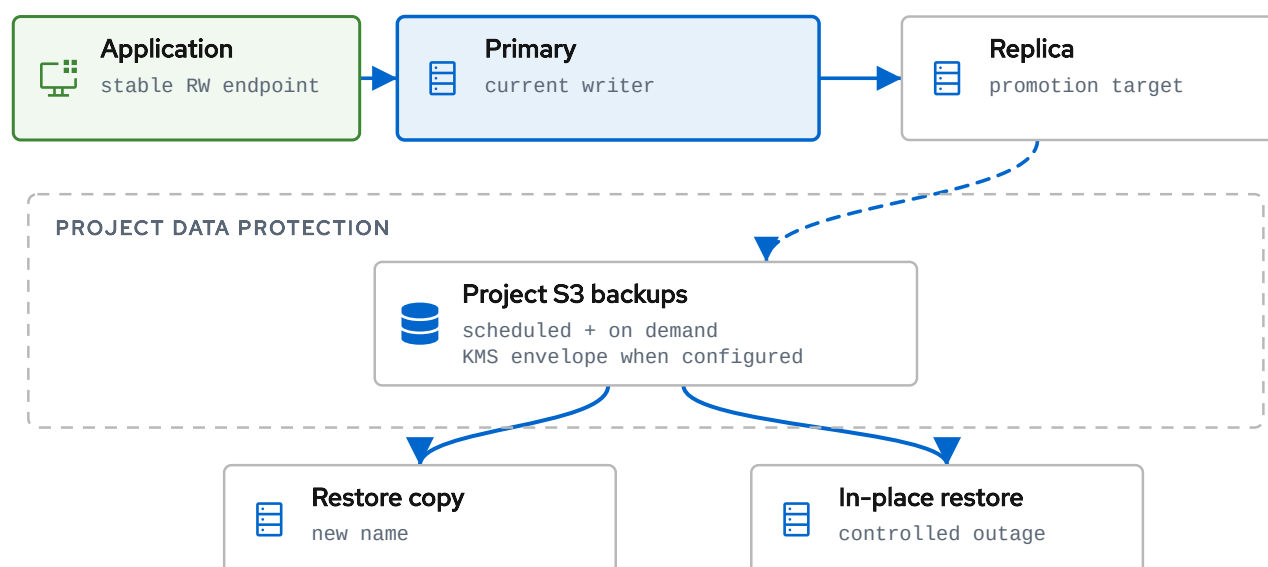
```
apiVersion: db.kube-dc.com/v1alpha1
kind: KdcDatabase
metadata:
  name: orders
  namespace: acme-shop
spec:
  engine: postgresql
  version: "16"
  databaseName: orders
  username: app
  cpu: "1"
  memory: 2Gi
  storage: 20Gi
  replicas: 2
  backup:
    enabled: true
    schedule: "0 2 * * *"
    retentionDays: 7
```

The platform provisions the engine, storage and credentials from the manifest, exposing a stable in-project endpoint (`orders-rw.<project>.svc`) for PostgreSQL; a primary-routing endpoint for MariaDB) and an auto-generated credentials Secret.

2. Topology and failover

- **1 replica** – standalone instance.
- **2+ replicas** – engine-level replication: PostgreSQL streaming replication; MariaDB primary-replica replication. With sufficient independent failure domains and capacity, instances schedule across separate hosts, and after an eligible primary failure the operator promotes an available replica automatically – recovery behavior depends on engine state and deployment topology, and the read-write endpoint follows the current primary.

SERVE · REPLICATE · BACK UP · RESTORE



Applications keep one read-write endpoint while engine replication and operator promotion protect service continuity; scheduled and on-demand backups land in Project S3 for new-name or in-place restore.

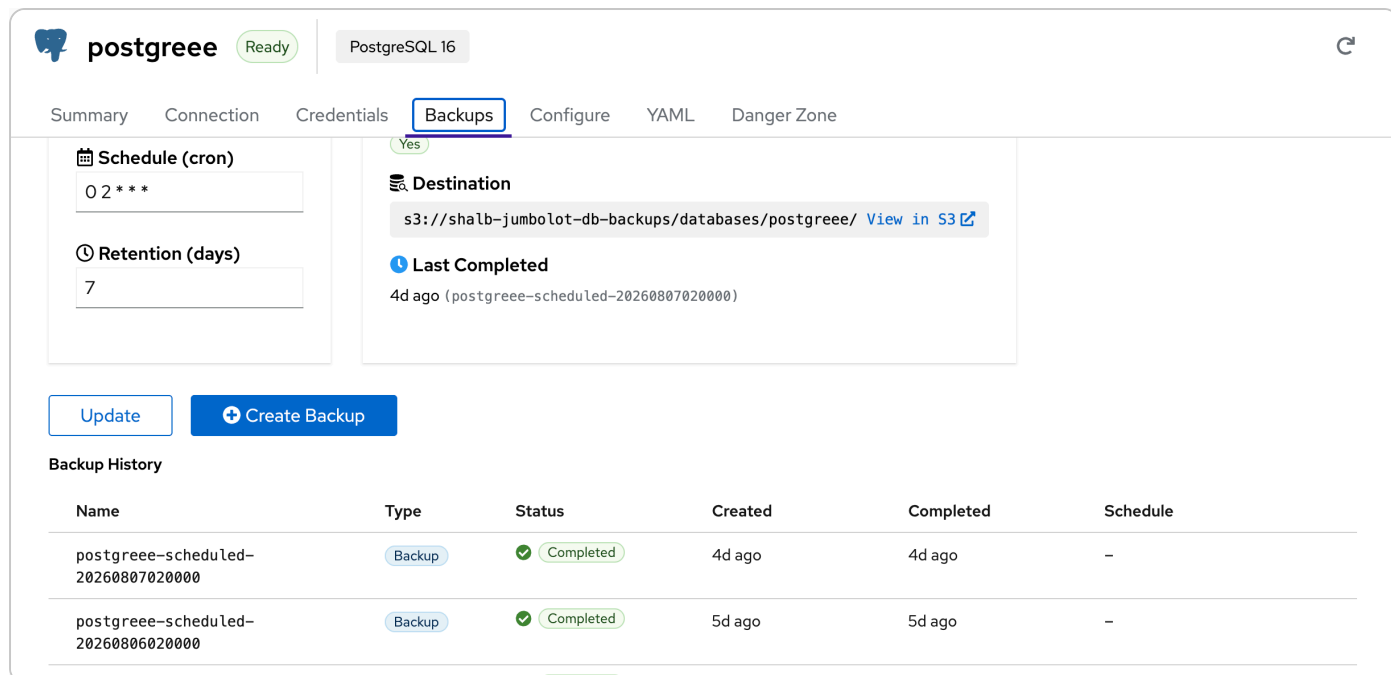
A practical note for multi-replica databases: the resource reports **Ready** when the service is available; check instance readiness before assuming full redundancy after creation or maintenance.

3. Credentials

- Connection details and passwords are generated by the platform and delivered as Kubernetes Secrets – applications consume them as environment variables or mounted files; nothing is typed or emailed.
- **Credential-rotation policies** rotate static passwords on a schedule and project the current credential into a stable Secret your workloads reference.

4. Backups

- **Scheduled backups** per database – cron schedule and retention days in the manifest – written to the project's S3-compatible bucket.
- **On-demand snapshots** any time, from the console or by creating a backup resource.
- **Per-database encryption**: reference a project KMS key and backups are envelope-encrypted with a key that never leaves the platform's secrets backend.
- For PostgreSQL, point-in-time recovery is available within the backup window where continuous archiving is enabled.



postgreee Ready PostgreSQL 16

Summary Connection Credentials **Backups** Configure YAML Danger Zone

Schedule (cron)
0 2 * * *

Retention (days)
7

Destination
s3://shalb-jumbolot-db-backups/databases/postgreee/ [View in S3](#)

Last Completed
4d ago (postgreee-scheduled-20260807020000)

[Update](#) [+ Create Backup](#)

Backup History

Name	Type	Status	Created	Completed	Schedule
postgreee-scheduled-20260807020000	Backup	✓ Completed	4d ago	4d ago	-
postgreee-scheduled-20260806020000	Backup	✓ Completed	5d ago	5d ago	-

Database protection is visible and operable from the service itself: schedule, retention, destination, on-demand backup and completion history share one view.

5. Restore

Two documented paths:

- **Restore into a new database** (recommended): create a new `KdcDatabase` with `spec.restoreFrom` naming the backup – verify the recovered data, then switch applications over. The source database keeps running.
- **In-place restore** (destructive): trigger by annotation on the existing database; the platform re-bootstraps it from the chosen backup. Plan a maintenance window – current data is replaced.

6. Configuration and lifecycle

- Resources (CPU, memory) and engine parameters adjustable per database; some changes restart instances or trigger failover – applications should reconnect and retry.
- Storage grows online (increase only).
- Version changes are maintenance operations: verify a current backup first, then move.

7. Responsibilities – managed databases

Concern	Your platform team	Tenant
Engine operation, replication, failover	☐ (controllers)	–
Backup execution and storage	Controllers execute scheduled backups	☐ schedule/retention choice, on-demand runs
Restore	Mechanism provided	☐ initiates, verifies recovered data
Credentials delivery and rotation	☐	☐ application usage hygiene
Schema, queries, application-level integrity	–	☐
Off-platform backup copies	S3 endpoints provided	☐ via enterprise backup

Networking & VLAN Attachment

Cloud networking semantics on your own fabric – VPCs, public endpoints and layer-2 reach into the datacenter you already run.

Kube-DC's network layer gives every project the isolation model of a public-cloud VPC, while staying connected to the physical reality of your datacenter: your uplinks, your address space, your existing VLANs.

1. Per-project VPCs

Every project receives its own VPC and private subnet at creation. No private cross-project route is configured by default – the boundary is enforced in the software-defined network, not by policy convention, and reachability between projects exists only where a service is deliberately exposed or the operator configures routing. Pods, VMs and managed-cluster workers in the same project share the project network and reach each other directly.

2. Address model

Object	What it does
Shared egress IP	Every project gets outbound connectivity through a shared external address
External IP (EIp)	A dedicated external address, allocated automatically per LoadBalancer service or explicitly for a purpose
Floating IP (FIP)	Maps an external address to an individual VM – inbound and outbound – without a load balancer

Address pools are defined by the platform team from your ranges; tenants consume them within quota.

3. Exposing services

Two documented paths, chosen per service:

- **HTTPS route (recommended for web/API/gRPC):** one annotation on a `LoadBalancer` Service publishes it at a hostname with a TLS certificate issued by the platform's configured issuer – no ingress controller for tenants to run, no certificate handling. Uses the platform's gateway, your routable address pools and DNS.
- **Direct LoadBalancer (any TCP/UDP):** the Service receives a dedicated external IP for protocols and ports beyond HTTP.

Managed-cluster API endpoints follow the same model: private in-VPC by default, public HTTPS through the platform gateway on request.

4. VLAN attachment: layer-2 into your datacenter

Where the datacenter fabric trunks delegated VLANs to the platform nodes and the operator configures the bridge and VLAN pools, a project's network can be bridged onto an existing datacenter VLAN.

- **Per-organization VLAN pools:** the platform team delegates ranges of VLAN IDs to organizations; tenants allocate segments from their pool self-service, without tickets and without being able to touch another organization's VLANs.
- Workloads – VMs and pods – gain a leg on the physical segment: lab instruments, storage networks, legacy systems, partner links, anything that expects layer-2 adjacency.
- Addressing on the attached segment is yours: static, your DHCP, or platform-managed.

VLANs

VLANs

Refresh

Segments (3)

Allocations (3)

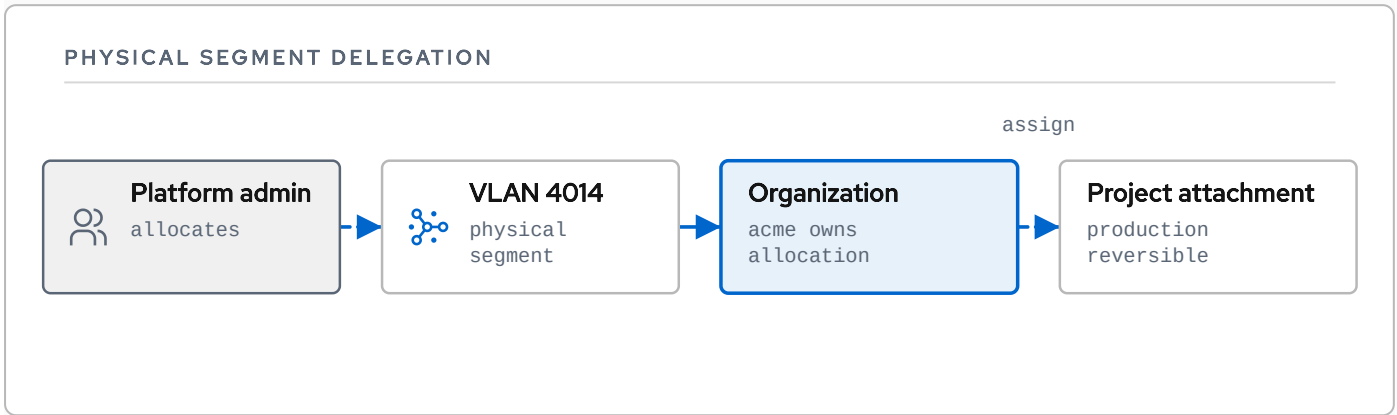
Assignments (2)

Which organization may use a wire. Their own admin then assigns it to one of their projects, and can unassign and re-assign it without a ticket.

Allocate to organization

Segment	Organization	Subnet	Gateway	State	Project	
pn-ext-cloud-1724 adopted	datapark	10.191.18.0/24	10.191.18.1	InUse	datapark/vlan-demo	
pn-ext-cloud-1734 adopted	datapark	10.191.48.0/24	10.191.48.1	InUse	datapark/demo	
pn-ext-cloud-1735	datapark	192.168.100.0/24	192.168.100.1	Allocated	unassigned	

Platform operators delegate physical segments to an Organization; its administrators can then assign or reassign those allocations to Projects without changing the underlying fabric.



Platform administrators allocate a physical segment to an Organization; Organization administrators then attach, detach, and later reattach that allocation to one of their Projects.

5. Traffic control

- Egress restrictions and allowlists are operator-configured platform controls – tenants request exceptions; the platform team owns the policy.
- Network boundaries are a property of the VPC layer: they are not tenant-configurable objects inside a project.

- North-south exposure happens through the objects above – an address or route a tenant deliberately created.

6. Responsibilities – networking

Concern	Your platform team	Tenant
Fabric, uplinks, address ranges, VLAN delegation	☐	–
VPC creation and isolation	☐ automatic	–
Egress policy and allowlists	☐	requests exceptions
Service exposure (LBs, routes, FIPs)	Mechanisms provided	☐ configures per service
VLAN segment allocation from the org pool	Pool delegation	☐ self-service within pool
DNS for published hostnames	Platform zone provided	☐ own domains via CNAME

Storage & Object Storage

Three storage services from the disks you rack – block, shared and S3 – consumed through standard Kubernetes APIs.

Kube-DC's storage layer is built on Ceph running on your hardware. The platform team defines the classes, topology and replication; tenants consume storage the Kubernetes-native way, within quota.

1. Block storage

- PersistentVolumes for containers and disks for VMs from a replicated Ceph RBD pool.
- Online volume expansion.
- Storage classes are operator-defined: you decide which tiers exist (replicated, node-local, performance) and what backs them – the platform runs on the disks and failure domains you give it.

2. Shared (read-write-many) storage

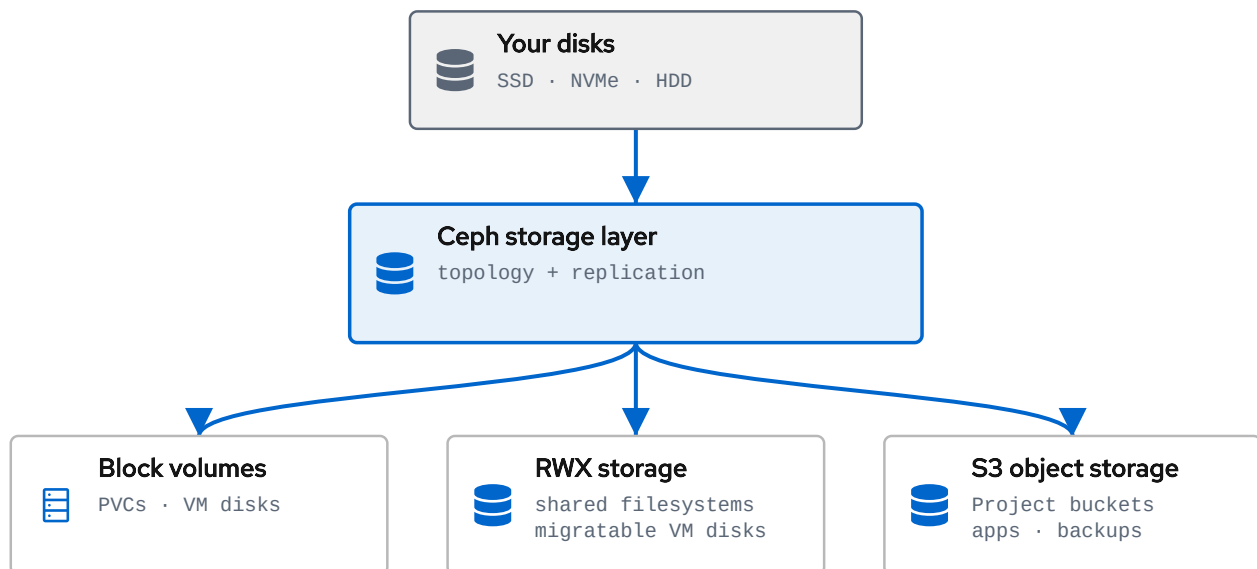
Operator-defined RWX storage classes serve the cases single-writer volumes can't – each with the access mode and backend appropriate to its semantics:

- **Multi-pod shared filesystems** – several pods writing the same volume (content stores, shared uploads, legacy apps that expect a shared disk).
- **Migration-eligible VM disks** – a VM whose disk is on the shared block class is eligible for live migration between hosts (see the Virtual Machines datasheet).

3. S3-compatible object storage

- **Per-project buckets** provisioned through a standard `ObjectBucketClaim` – credentials arrive as a Secret and ConfigMap the workload mounts; no manual account management.
- The platform's own services use the same storage: database backups and managed-cluster snapshots land in per-project buckets.
- Typical tenant uses: application assets, exports, log archives, backup targets.

PHYSICAL CAPACITY · STANDARD CONSUMPTION



The operator's disk topology feeds Ceph, which publishes block, shared and S3 services to Project workloads through standard Kubernetes storage interfaces.

4. Snapshots, clones and images

- **Volume snapshots** through the standard Kubernetes snapshot API – point-in-time copies of workload and VM volumes.
- **Instant clones**: new volumes from snapshots without duplicating data – the mechanism behind the prepared-OS-image catalog and VM provisioning without image downloads.
- Prepared images are published per project as snapshots.

5. Capacity and quota

Storage consumption counts against organization plans and project quotas like CPU and memory. Object-storage capacity is governed per organization and requires its own capacity planning. How many disks, which tiers, what replication – these are platform-team decisions made against your hardware; capacity and durability are properties of the topology your team designs, validated in the architecture review.

6. Data protection posture

Storage replication protects against component failure within the platform; it is not a backup. The protection stack is layered:

Layer	Mechanism	Owner
Component failure	Ceph replication per your topology	Platform team
Point-in-time copies	Scheduled jobs run by platform controllers; on-demand snapshots and restores initiated by tenants	Shared
Site/storage loss	Copied by your configured enterprise backup integration via S3 endpoints	Your backup team

7. Responsibilities – storage

Concern	Your platform team	Tenant
Disks, tiers, replication, failure domains	☐	–
Storage classes and quotas	☐ defines	consumes within quota
Volume/bucket provisioning	Automatic via API	☐ requests
Snapshots and clones	APIs provided	☐ initiates
Backup of application data	S3 + snapshot APIs provided	☐ owns

Security, Identity & Keys

Tenancy enforced by mechanisms, not conventions – from sign-on to the network to the keys.

Kube-DC layers organization identity, project RBAC, VPC network boundaries and project-scoped keys and secrets as defense in depth – each boundary enforced by a distinct system.

1. Identity and single sign-on

- **An identity realm per organization** (Keycloak): each tenant organization manages its own users, groups and login policies – isolated from every other organization's.
- **Single sign-on across every surface:** console, Kubernetes API and CLI authenticate against the same realm via OIDC. Platform-project kubeconfigs use organization OIDC credentials; managed-cluster admin/break-glass kubeconfigs use separate per-cluster credentials.
- **Group-based access:** organization groups map to project roles declaratively, so joiners/leavers are handled in the directory, not in per-project role edits.

Edit Organization Group

jumbolot

Roles *

Select All

Deselect All

☐ admin

☐ default-project-admin-role

☐ default-project-developer-role

☐ default-project-manager-role

☐ default-project-user-role

☒ developer

☐ kccm-boom

Project Permission 2

Remove

Project *

docs

Roles *

Select All

Deselect All

☐ admin

☐ developer

☐ project-manager

☒ user

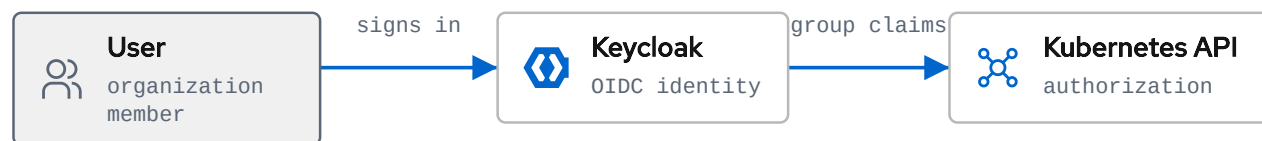
Update Group

Cancel

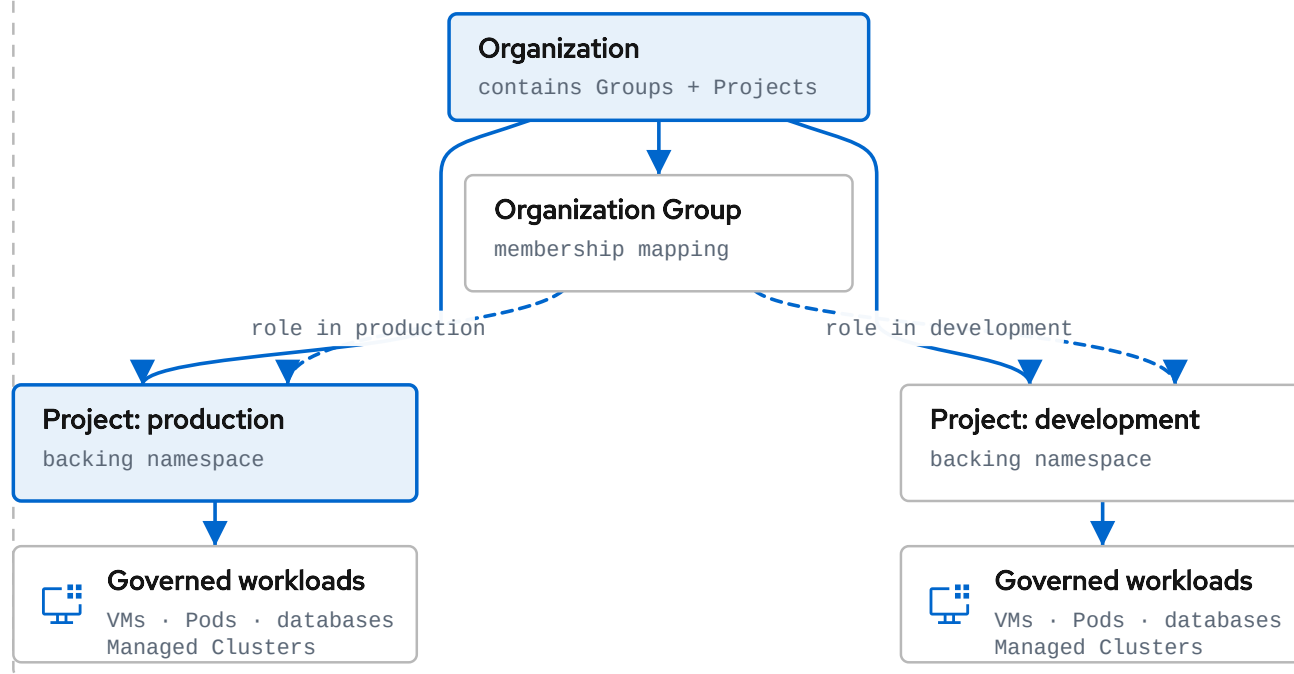
Organization Groups map identity roles to Project-specific permissions, keeping access assignments explicit at each governed workload boundary.

37 / 44

IDENTITY PATH



ORGANIZATION · GOVERNANCE BOUNDARY



Identity claims establish API access, while Organization Groups map roles independently into each Project; workloads and Managed Clusters remain governed by their authorized Project.

2. Project authorization

- **A curated, supported role set** per project – scoped to deploying and operating workloads. Cluster-scoped resources, CRDs and admission configuration are excluded by construction.
- **Interactive `exec/attach` into pods is excluded from the supported project roles** and denied by an admission policy at the API. One-off tasks run as auditable Jobs instead. Scope stated plainly: users permitted to create or modify workloads can still run code through those workloads, and the restriction does not apply inside tenant-administered managed clusters.
- Custom roles remain under platform-team control; the default roles are a baseline, not a ceiling.

3. Key management (KMS)

- **Per-project, purpose-scoped keys** (`application`, `backup`, ...) backed by non-exportable keys in the platform's secrets backend (OpenBao Transit): encrypt/decrypt happens as an API operation – key material never leaves the backend.
- Used by the platform itself where it matters: managed-database backups and managed-cluster snapshot encryption reference tenant KMS keys.
- Envelope encryption is the documented pattern for large payloads.

4. Secrets management

- **A managed secret service per project:** store API tokens, credentials and configuration secrets outside Git, with lifecycle managed through the CLI and console.
- **Sync into Kubernetes Secrets:** selected values project into standard Secrets that workloads consume – applications stay platform-agnostic while the source of truth stays managed.

5. Certificates

- **Managed certificates** as a project resource: public trust via ACME or issuance from your organization's private CA – the resulting TLS Secrets are consumed by workloads directly.
- HTTPS routes get their certificates automatically (see the Networking datasheet); managed-cluster public endpoints likewise.

6. Platform-level enforcement

- Admission policies enforce tenant boundaries at the Kubernetes API – including the pod-access block and resource-placement rules.
- Network isolation is a property of the SDN (per-project VPCs), not of tenant-managed policy.
- Egress control and allowlists are operated centrally by the platform team.
- Platform configuration can be declaratively reconciled from Git.

7. Responsibilities – security & identity

Concern	Your platform team	Tenant
Identity realms, SSO, login policy defaults	☐ provisions	☐ manages own users/groups
Project roles and admission policies	☐	assigns members to roles
KMS backend and key custody	☐	☐ creates/uses keys
Secrets service	☐ operates	☐ owns contents
Certificates	Issuance machinery	☐ requests/consumes
Workload and guest-OS security	–	☐

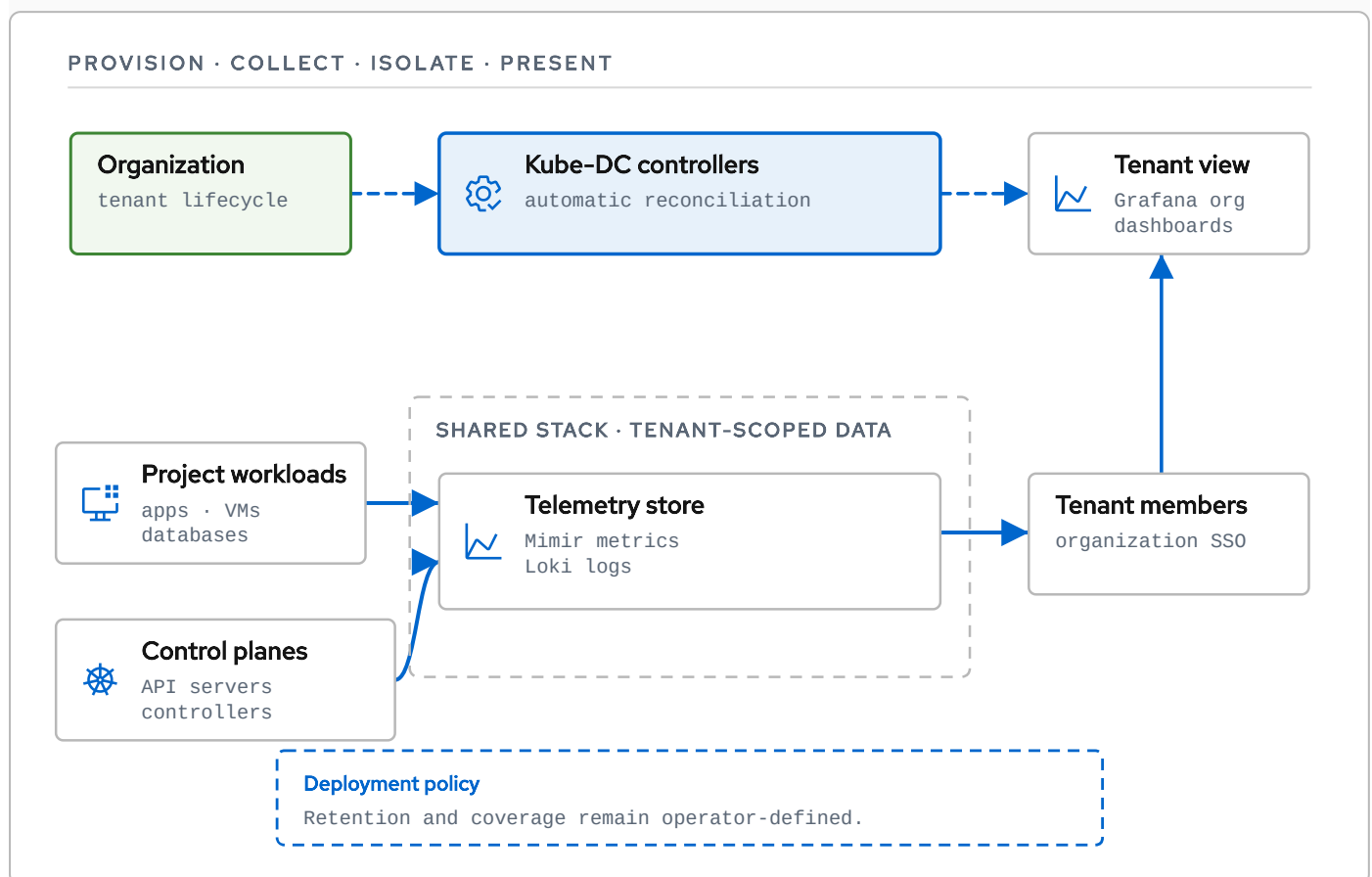
Observability

Every organization gets monitoring on day one – logs, metrics and dashboards, configured automatically. Nobody builds a monitoring stack per team.

Kube-DC ships a multi-tenant observability stack (Grafana, Mimir for metrics, Loki for logs) as part of the platform. When an organization is created, the platform's controllers provision its observability automatically: a Grafana organization of its own, pre-built dashboards, a metrics tenant and log routing. Teams open Grafana and see their workloads – there is nothing to install, wire or configure.

1. What every organization gets

- **A Grafana organization of its own** – members sign in with the same SSO they use everywhere else and land in their organization's view.
- **Pre-provisioned dashboards** for the organization's projects and workloads – created and maintained by the platform's controllers, not hand-built per team.
- **Metrics** collected from the organization's workloads and served from the multi-tenant metrics store, scoped per tenant.
- **Logs** from the organization's workloads, routed per tenant and queryable from the same Grafana.
- **Managed-cluster control-plane telemetry**: the logs and events of an organization's managed Kubernetes clusters surface in its Grafana – API servers and controllers are watchable without filing a ticket.



Organization reconciliation provisions Grafana, metrics tenancy and log routing; workloads and managed control planes feed the shared data layer, while tenant-scoped queries return only that organization's data.

2. Tenant separation

The stack is shared; the data is not. Metrics and logs are separated per tenant at the data layer – queries from one organization's Grafana reach only that organization's data. The platform operates one observability system for all tenants instead of one stack per team, which is what makes day-one monitoring economically automatic.

3. The operator's view

The platform team monitors the platform itself from the same tooling:

- Cluster health, capacity, networking and component status, with alerting for platform operators.
- **Storage health:** Ceph dashboards and alerts for the storage layer, surfaced in the operator's monitoring and the administration console.
- Coverage and retention are operator-defined – the bundled defaults are a starting point, tuned to your capacity in the architecture review.

4. Responsibilities – observability

Concern	Your platform team	Tenant
Observability stack operation and capacity	☐	–
Per-organization provisioning (Grafana org, dashboards, tenants, routing)	☐ automatic, by controllers	–
Retention and coverage configuration	☐	–
Watching workload dashboards, acting on application signals	–	☐
Custom dashboards within the organization's Grafana	–	☐
Platform alerting and response	☐	reports suspected platform issues

GPU Services

Two ways to give teams accelerators – a fraction of a card, or a whole one – with the governance an organization needs to hand GPUs to departments.

GPUs are scarce, expensive and shared badly by default. Kube-DC treats them as a product: catalogued profiles, per-model entitlements enforced as quota, capacity reservations for organizations that need them, and an explicit security and supply-chain model – because the components that make GPUs work run privileged on your nodes.

GPU capabilities are enabled per cluster by your platform team once the hardware, driver and component set are qualified – GPU nodes couple device, kernel, driver and operator versions, so enablement follows qualification rather than a switch flip. Shared GPU and dedicated GPU VM creation are separately controlled, so a deployment can offer one without the other.

1. Shared GPU for containers

Several container workloads run on one physical GPU at the same time, each holding a fixed fraction of the device – the right fit for inference, notebooks, small training runs and batch scoring, where a whole card is waste.

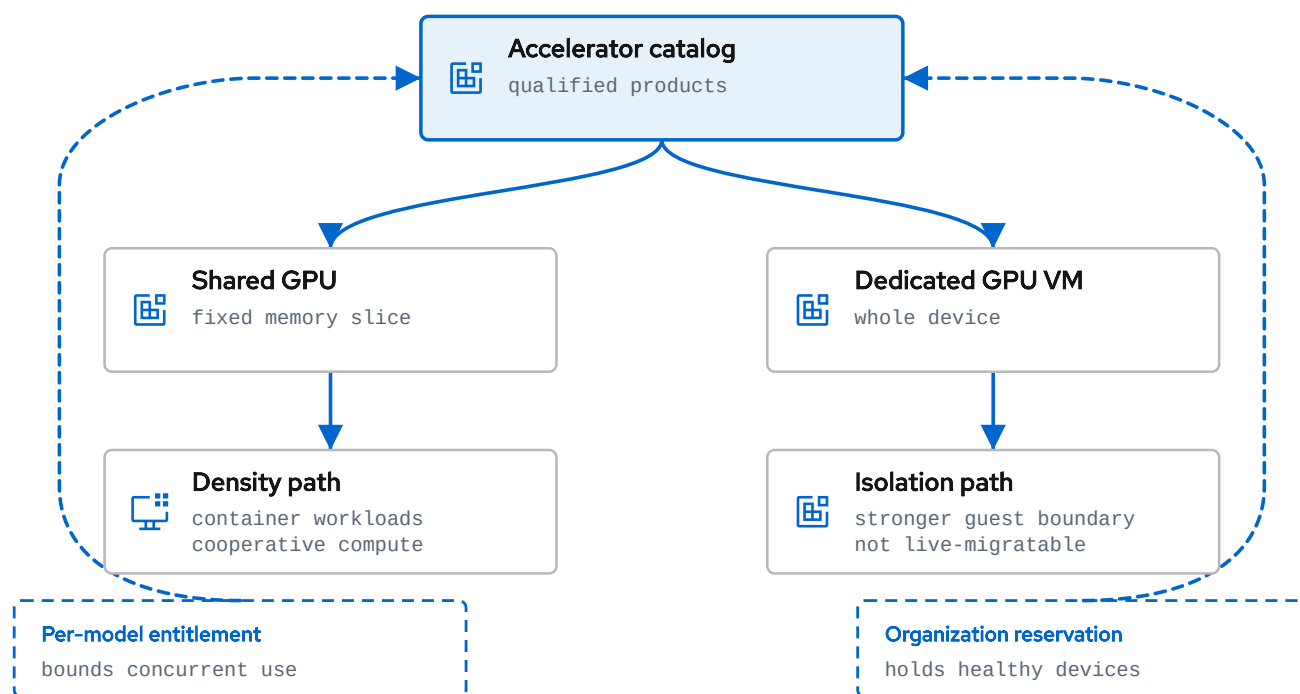
- **Fixed-product profiles.** A shared product is a defined fraction of a specific GPU model (for example, an 8 GiB slice of an NVIDIA V100) published in the platform's accelerator catalog. Tenants request a product; they do not freely tune memory and compute.
- **The memory slice is enforced.** Inside the container, the GPU reports the slice size, not the physical card.
- **The compute share is cooperative.** Workloads are steered toward their share in steady state; startup and some library paths can briefly exceed it. Stated plainly: this is a density mechanism, **not a performance guarantee and not a security boundary** – for a hard boundary, use a dedicated GPU VM.
- **Standard Kubernetes consumption.** Workloads request GPU products through Kubernetes' Dynamic Resource Allocation – a manifest applied with `kubect l`, like any other workload. Shared GPU runs tenant workloads in production on the reference deployment today.

2. Dedicated GPU VMs

One whole physical GPU attached to one virtual machine, for workloads that need a stronger boundary than software sharing: an isolated guest with the full device.

- Whole-device passthrough to a Linux or Windows guest, from the deployment's qualified image and driver matrix.
- **Not live-migratable** – planned maintenance means shutdown and restart, and a later start can wait for capacity or receive a different physical device.
- Guest images and driver packages are qualified per deployment: the platform team publishes an approved package version and checksum before enabling GPU VM creation. The published matrix names the qualified guests; anything outside it is qualified separately.

CATALOG · ENTITLEMENT · CAPACITY



The accelerator catalog offers a density-oriented shared slice for containers and a whole-device VM path; model-specific quota bounds use, while a separate Organization reservation holds healthy capacity.

3. Entitlements, quota and reservations

Two different things, deliberately distinguished:

- **Entitlement (quota)** – each GPU model is metered independently in organization and project quota, so a team's V100 allowance never consumes another model's headroom. An entitlement bounds concurrent use; it does **not** reserve a physical device, so a valid workload can legitimately queue when all matching GPUs are busy.
- **Capacity reservation** – an operator-held amount of healthy physical capacity set aside for one organization, tracked in a separate capacity ledger. This is what a department buys when queuing is unacceptable. Reservations are fungible: the guarantee is the number of healthy devices, not a specific card.

Assigning a GPU add-on does not by itself create a reservation – the distinction is enforced in the product, not just in the documentation.

4. Node modes and day-2 operations

A GPU node runs in one mode at a time – shared-container or dedicated-VM passthrough – because the two require different host configuration.

- **Holder-safe mode transitions:** the `kube-dc` CLI performs the day-2 switch between modes with an explicit safety contract – creation gates closed, GPU holders inventoried, node cordoned, fleet state and live state in agreement before anything moves.

- **Upgrade gating:** GPU nodes couple the PCI device, host kernel, Kubernetes distribution, driver, GPU operator and monitoring exporter into **one tuple** that must be validated together – upstream support for each component separately is not sufficient. The CLI provides a read-only pre-upgrade gate; the documented sequence qualifies a canary node and proves rollback before a fleet-wide plan.

5. Security and supply chain

GPU drivers, device plugins, schedulers and monitoring agents run privileged enough to compromise a node, so version selection and image provenance are part of the security boundary – not routine application upgrades.

- **A published threat model** covers shared containers, dedicated VMs, catalog and discovery, quota, billing, installation, node-mode transitions, monitoring and the user interfaces – including the invariants the design preserves (a tenant cannot mint GPU entitlement by editing ordinary resources).
- **A qualified installer tuple:** an enabled installation accepts one indivisible, version-pinned component set, with runtime audit and a promotion gate – not "latest".
- Admission policies force tenant GPU requests into the exact catalog product, and per-model quota bounds the count, so entitlement cannot be exceeded by hand-written manifests.

6. Responsibilities – GPU services

Concern	Your platform team	Tenant
Hardware, drivers, GPU operator, qualification of the component tuple	☐	–
Enabling GPU capabilities per cluster and per project	☐ (gated, after qualification)	–
Catalog products (which slices and models exist)	☐ defines	☐ requests
Entitlements and reservations	☐ grants	☐ consumes within quota
Node-mode transitions and GPU node upgrades	☐	–
Workload design, drivers inside VM guests, performance tuning	–	☐